

1. ¿Qué es la programación dirigida por eventos?

👉 El programa **espera a que ocurra algo** (evento)

Ejemplos de eventos:

- Clic en un botón
- Escribir en un campo
- Cerrar ventana

💡 En lugar de ejecutar todo en orden, el programa:

👉 **reacciona**

2. Conceptos clave

- **Evento** → lo que ocurre (clic, teclado...)
 - **Fuente (source)** → el componente (botón, ventana...)
 - **Listener** → quien escucha el evento
-

3. Primer ejemplo: ventana básica (Swing)

```
import javax.swing.JFrame;
```

```
public class VentanaBasica {  
    public static void main(String[] args) {  
        JFrame ventana = new JFrame("Mi primera ventana");  
  
        ventana.setSize(400, 300);  
        ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        ventana.setVisible(true);  
    }  
}
```

Explicación:

- `JFrame` → ventana
 - `setSize()` → tamaño
 - `setVisible(true)` → mostrar
-

4. Añadir un botón

```
import javax.swing.*;

public class VentanaConBoton {
    public static void main(String[] args) {
        JFrame ventana = new JFrame("Ejemplo");

        JButton boton = new JButton("Haz clic");

        ventana.add(boton);

        ventana.setSize(300, 200);
        ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        ventana.setVisible(true);
    }
}
```

5. Manejar eventos (ActionListener)

👉 Aquí empieza lo importante

```
import javax.swing.*;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;

public class EventoBoton {
    public static void main(String[] args) {
        JFrame ventana = new JFrame("Eventos");

        JButton boton = new JButton("Pulsa");

        boton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                System.out.println("Botón pulsado");
            }
        });

        ventana.add(boton);

        ventana.setSize(300, 200);
        ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        ventana.setVisible(true);
    }
}
```

6. Versión moderna (lambda)

```
boton.addActionListener(e -> {  
    System.out.println("Botón pulsado");  
});
```

- ✓ Más limpio
 - ✓ Muy usado hoy en día
-

7. Mostrar mensajes (JOptionPane)

```
import javax.swing.JOptionPane;
```

```
JOptionPane.showMessageDialog(null, "Hola alumno");
```

8. Ejemplo completo (MUY importante)

👉 Ventana con botón que muestra mensaje

```
import javax.swing.*;
```

```
public class AppSimple {  
    public static void main(String[] args) {  
        JFrame ventana = new JFrame("App");  
  
        JButton boton = new JButton("Saludar");  
  
        boton.addActionListener(e -> {  
            JOptionPane.showMessageDialog(null, "Hola!");  
        });  
  
        ventana.add(boton);  
  
        ventana.setSize(300, 200);  
        ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        ventana.setVisible(true);  
    }  
}
```

9. Añadir campo de texto

```
import javax.swing.*;
```

```
public class CampoTexto {
```

```
public static void main(String[] args) {
    JFrame ventana = new JFrame("Texto");

    JTextField campo = new JTextField(15);
    JButton boton = new JButton("Mostrar");

    boton.addActionListener(e -> {
        System.out.println(campo.getText());
    });

    JPanel panel = new JPanel();
    panel.add(campo);
    panel.add(boton);

    ventana.add(panel);

    ventana.setSize(300, 200);
    ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    ventana.setVisible(true);
}
}
```

10. Layouts (organizar componentes)

👉 Sin layout → todo se ve mal

Ejemplo con FlowLayout (por defecto)

JPanel panel = new JPanel(); // FlowLayout automático

Otros layouts (solo mencionar a alumnos):

- BorderLayout
 - GridLayout
-

11. Tipos de eventos (teoría importante)

- `ActionEvent` → botones
 - `WindowEvent` → ventana
 - `MouseEvent` → ratón
 - `KeyEvent` → teclado
-

12. Ejemplo completo con lógica

👉 Mini app: sumar números

```
import javax.swing.*;

public class Calculadora {
    public static void main(String[] args) {
        JFrame ventana = new JFrame("Calculadora");

        JTextField num1 = new JTextField(5);
        JTextField num2 = new JTextField(5);
        JButton boton = new JButton("Sumar");

        boton.addActionListener(e -> {
            int a = Integer.parseInt(num1.getText());
            int b = Integer.parseInt(num2.getText());

            int resultado = a + b;

            JOptionPane.showMessageDialog(null, "Resultado: " + resultado);
        });

        JPanel panel = new JPanel();
        panel.add(num1);
        panel.add(num2);
        panel.add(boton);

        ventana.add(panel);

        ventana.setSize(300, 200);
        ventana.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        ventana.setVisible(true);
    }
}
```

13. Buenas prácticas

- ✓ Separar lógica y GUI (más adelante MVC)
- ✓ Usar lambdas
- ✓ Validar datos (`parseInt` puede fallar)
- ✓ No saturar el `main`

EJERCICIOS

1. Ventana básica

Crear JFrame simple.

2. Botón que imprime mensaje

Evento ActionListener.

3. Botón con lambda

Versión moderna.

4. Mostrar JOptionPane

Mensaje emergente.

5. Campo de texto + botón

Mostrar texto en consola.

6. Saludo personalizado

Escribir nombre y mostrar saludo.

7. Calculadora suma

Dos campos + resultado.

8. Calculadora completa

Suma, resta, multiplicación.

9. Contador de clics

Botón que incrementa contador.

10. Mini app completa

Formulario:

- nombre
 - edad
- Botón → mostrar datos en ventana.